

FarmOS 智能体 MCP 接入指南

本项目中涉及到的所有智能体均通过 MCP (Model Context Protocol) 标准来对接调度中枢。本指南将帮助各组开发者快速理解和实现 MCP 服务。

1. MCP 简介

MCP 是一种标准化的通信协议，旨在使大型语言模型能够与外部工具和数据源无缝交互。

1.1 基础概念

- **Server**：提供工具、数据资源或提示词的服务方。在我们的架构中，各组开发的智能体即为 Server。
- **Client**：调用 Server 提供的能力的一方。在本项目中，FarmOS 的调度中枢作为 Client。

1.2 MCP 的三大核心原语

MCP 主要提供以下三种能力：

1. **Tools**：允许 LLM 执行外部操作（如查询天气、执行计算、控制硬件等）。
2. **Resources**：允许 Server 向 LLM 提供只读数据内容（如文件、数据库记录等）。
3. **Prompts**：允许 Server 提供预设的提示词模板。

当前实践指引：我们的项目目前只使用 MCP 的 **Tool (工具)** 能力。各组只需要将智能体的功能封装为 Tool 暴露出来即可。

1.3 MCP 通信方式

MCP 本质上是一套交换 JSON 格式消息的标准，它可以运行在不同的底层传输协议上。主要有三种：

1. **stdio**：本地进程间的通信方式，常用于本地 CLI 工具。
2. **HTTP Streamable**：基于 URL 的网络访问，适用于分布式系统。这是本项目 MCP 服务实现的通信方式。
3. **SSE**：一种单向事件流技术，目前已经不推荐使用。

2. 对接目标与设计原则

各组的最终目标是提供一个或多个 **ASGI 应用程序**（**demo.py已经实现了**），作为提供 MCP 接口的 Web 服务。

2.1 服务划分原则

- 各组需要自行启动服务，并最终向调度中枢提供可访问的接口 URL。
- **Server 数量**：由各组根据智能体之间的关联程度自行裁定。一般来说，将同属于一个工程模块的所有智能体工具暴露在同一个 Server 实例中是最简洁的。
- 对于调度中枢而言，Server 的数量无关紧要，因为经过解析后，所有 Server 下的方法都会被平铺整合到一个统筹的工具列表中供大模型调用。

2.2 异步与性能注意事项

- 如果流程中涉及到阻塞等待（如网络请求、IO 等耗时操作），可以考虑异步函数 (`async def`) 的方式来实现工具，以避免阻塞 FastMCP 服务的事件循环。

2.3 工具参数说明规范

为了让模型更准确地理解工具入参，建议为每一个参数补充文本说明。推荐使用 Python 的 `Annotated`，官方文档的例子如下：

```
from typing import Annotated

@mcp.tool
def process_image(
    image_url: Annotated[str, "要处理的图像的 URL"],
    resize: Annotated[bool, "是否调整图像大小"] = False,
    width: Annotated[int, "目标宽度 (像素)"] = 800,
    format: Annotated[str, "输出图像格式"] = "jpeg"
) -> dict:
    """使用可选调整大小处理图像。"""
```

`demo.py` 已按该规范为每个工具参数补充了说明，可直接参考并改造为组内业务参数。

2.4 模型返回类型说明

在 MCP 工具设计中，返回值并不局限于单一形式。可以根据业务实际情况，选择使用结构化 JSON 返回内容，或作为 Markdown 文本返回。`demo.py` 中的 `generate_task_report_markdown` 是一个返回 Markdown 的例子。

3. 代码示例与调试方法指南

使用 `fastmcp` 库来快速构建和暴露 MCP 服务。我们提供了一个参考程序 `demo.py`。

最终，各组只需将部署好的 **URL 地址** (如 `http://<ip>:<port>/mcp`) 提供给调度中枢模块即可。

在向最终项目交付前，各组可以使用 MCP 官方提供的 Inspector 工具进行自测。

1. 启动你的目标服务：

确保已安装 uvicorn：`pip install uvicorn`。

使用 pip 安装 `demo.py` 的依赖 `fastmcp` 和 `starlette`（`starlette` 用于开发调试配置跨域中间件，生产环境不需要）

然后使用类似如下的命令启动（在代码示例中有具体说明）：`uvicorn demo:app --host 0.0.0.0 --port 8000`。

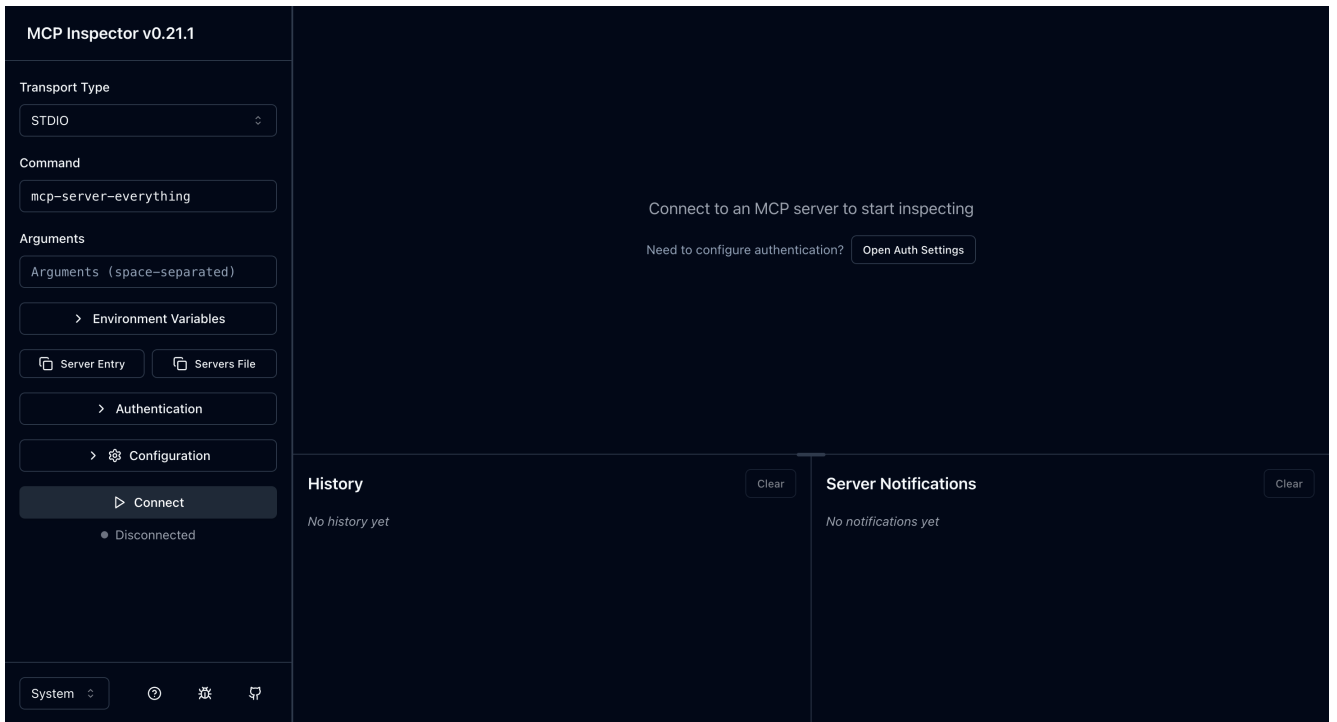
2. 启动 MCP Inspector：

确保本机安装了 Node.js，运行以下命令启动 Inspector：

```
npx @modelcontextprotocol/inspector
```

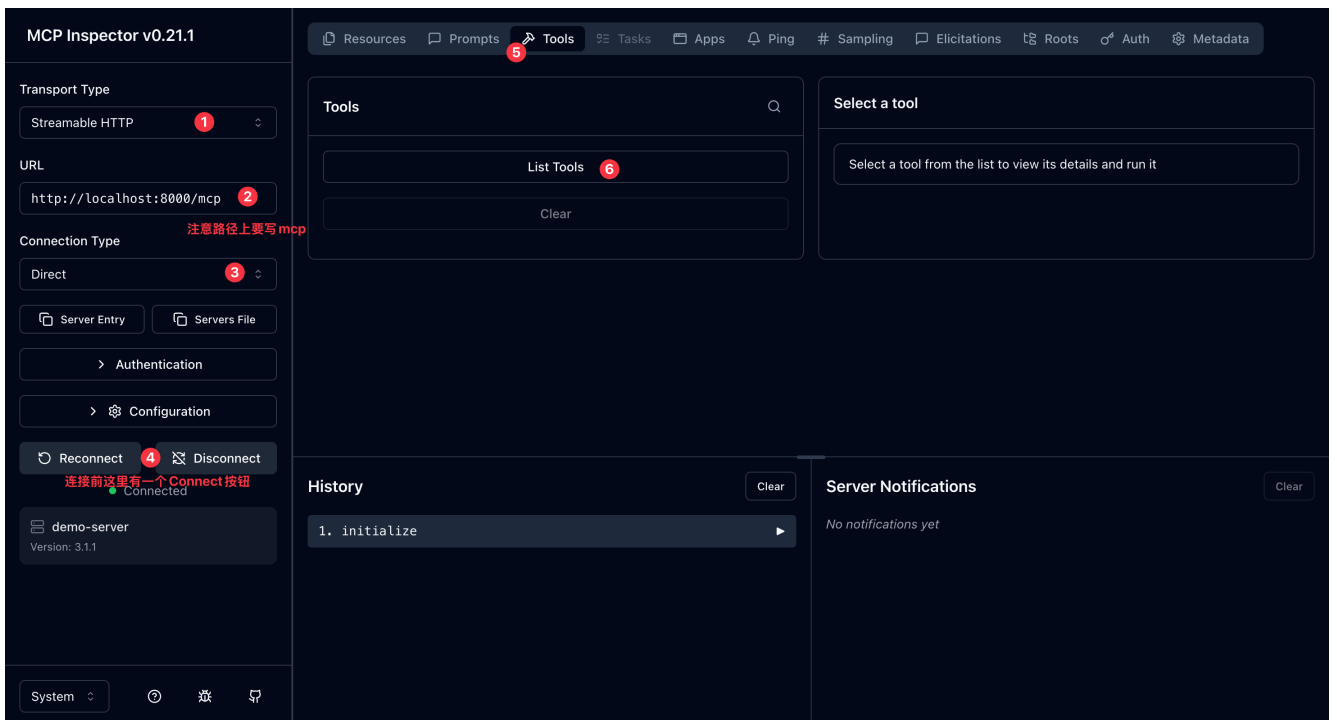
3. 访问调试器网页：

默认情况下，Inspector 会在本地浏览器打开一个调试界面（如 `http://localhost:6274`）。



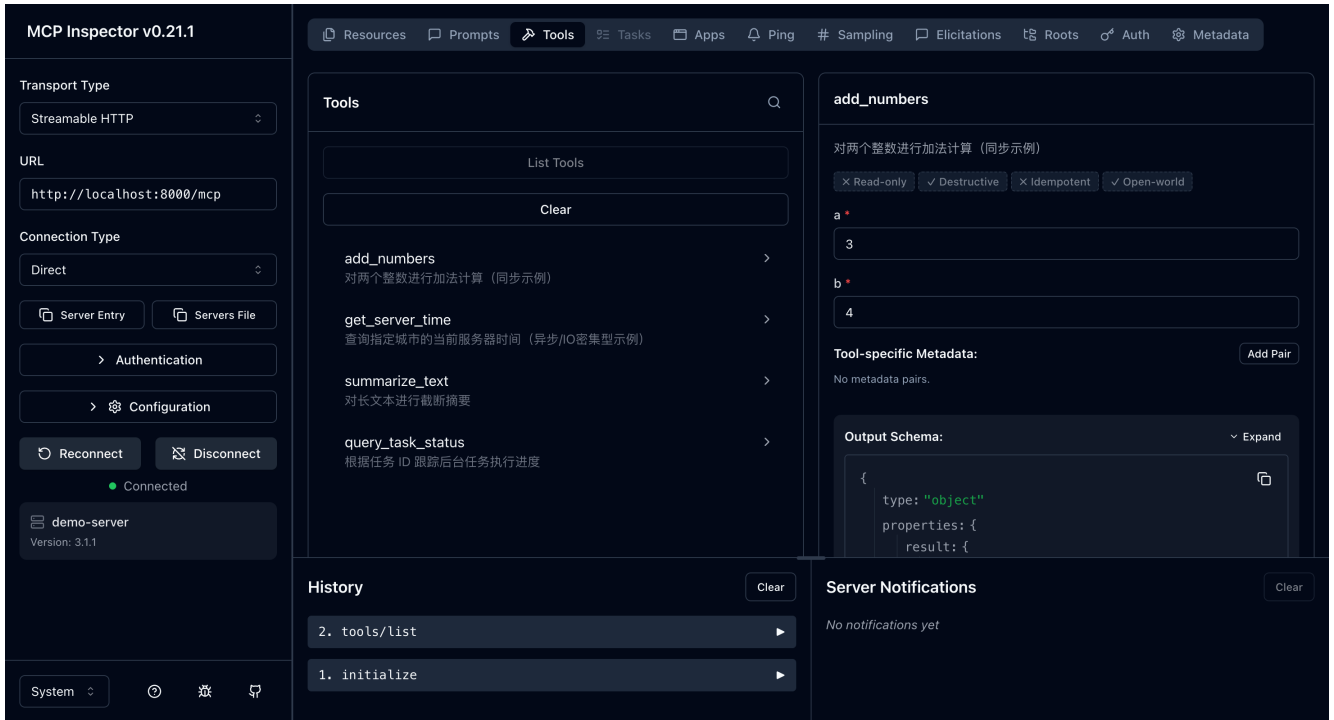
4. 建立连接：

- **Transport Type**：选择 `Streamable HTTP`。
- **URL**：填写为 `http://localhost:8000/mcp`。**注意：一定要加上 `/mcp` 路径！** 否则无法正确建立 SSE 或 HTTP 连接。
- **Connection Type**：确保是 `Direct`。
- 点击 **Connect**。
- 连接成功后，在顶部选择 **Tools** 标签，然后点击 `List Tools` 获取已暴露的工具列表。



5. 开始调试：

在左侧工具列表中找到你的函数，右侧可以输入测试参数并发起请求 (Run Tool)，检查返回结果是否符合预期。



备注：

1. 远程服务器调试：

如果项目位于远程服务器且难以直接暴露调试端口，可以使用 VS Code 的端口映射功能。
通过 SSH 连接到远程服务器后，将内部开放端口（如 8000）转发到本地机器的 localhost:8000，然后在本地电脑运行 npx @modelcontextprotocol/inspector 并连接 http://localhost:8000/mcp 即可调试。

2. 前期建议：

建议先运行本指南提供的 demo.py 跑通 Inspector 连接。理解整个通路的交互流程后，再在此基础上开发组内业务逻辑。

4. 参考文档

- FastMCP 工具文档（中文）：<https://fastmcp.wiki/zh/servers/tools>
- MCP Inspector（用于本地联调）：<https://github.com/modelcontextprotocol/inspector>